

Document 12 — Platform Operations, Deployment & DevOps Blueprint

Product Name: HQ

Tagline: Your AI Headquarters

Version: 1.0

Purpose

This document defines how HQ will be deployed, operated, monitored, secured, and maintained throughout its lifecycle.

The deployment strategy prioritizes reliability, scalability, developer productivity, and operational simplicity while leveraging Google Cloud Platform (GCP) for the hackathon deployment.

The cloud provider does not define HQ's identity. It is simply the infrastructure that powers the platform.

Platform Principles

The HQ platform must be:

- Reliable
- Scalable
- Secure
- Observable
- Cost-efficient
- Maintainable
- Cloud-ready
- Provider-independent at the application level

Environment Strategy

HQ operates in three environments.

Development

Purpose:

- Local development
- Feature implementation
- Debugging

Infrastructure:

- Docker
- Local PostgreSQL
- Local Redis
- Environment Variables

Staging

Purpose:

- Internal testing
- QA
- User acceptance testing

Infrastructure mirrors Production as closely as possible.

Production

Purpose:

Serve real users with high availability and monitoring.

Cloud Infrastructure

The MVP will be deployed on Google Cloud Platform (GCP).

Core services include:

- Cloud Run
- Cloud SQL (PostgreSQL)
- Cloud Storage
- Secret Manager
- Cloud Logging
- Cloud Monitoring

This infrastructure satisfies hackathon requirements while remaining suitable for future commercial deployment.

Application Deployment

Frontend

Technology:

- Next.js

Deployment:

- Firebase App Hosting (preferred)
- Cloud Run (alternative)

Backend

Technology:

- NestJS

Deployment:

- Docker Container
- Cloud Run

Cloud Run provides automatic scaling and simplified operations for the MVP.

Database

Primary database:

PostgreSQL

Deployment:

Cloud SQL

Responsibilities:

- Users
- Companies
- Headquarters
- Missions
- Assets
- Billing
- Analytics

Object Storage

Cloud Storage stores:

- Images
- Videos
- Documents

- Brand Assets
- Generated Content
- User Uploads

Database records store metadata only.

Secret Management

Sensitive information is never stored in source code.

Examples:

- API Keys
- Database Credentials
- JWT Secrets
- OAuth Credentials
- Payment Keys

Managed using Google Secret Manager.

Container Strategy

Every deployable service runs inside Docker.

Benefits:

- Consistent environments
- Simplified deployment
- Easy scaling
- Reproducible builds

CI/CD Pipeline

Source Control:

GitHub

Pipeline:

Developer Push

↓

Automated Checks

↓

Lint



Type Check



Tests



Docker Build



Deploy to Staging



Manual Approval



Production Deployment

Database Migration Strategy

Every schema change is version controlled.

Migration workflow:

- Create Migration
- Review
- Test
- Deploy
- Verify

Database changes must never be performed manually in production.

Backup Strategy

Automatic backups include:

- Database
- Uploaded assets
- Configuration

Backups should be encrypted and regularly tested through restoration exercises.

Monitoring

Platform monitoring includes:

- API availability
- Response times
- Error rates
- Database health
- Container health
- AI request latency
- Storage utilization

Operational visibility is essential for maintaining service quality.

Logging

Application logs include:

- Authentication events

- API requests
- Mission execution
- AI interactions
- Errors
- Deployment history

Logs must exclude sensitive user data.

Alerting

Alerts are generated for:

- Service downtime
- Failed deployments
- Database connectivity issues
- Elevated error rates
- High resource usage

Critical alerts should notify the engineering team immediately.

Security Operations

Operational security includes:

- HTTPS enforcement
- Secret rotation
- Dependency updates
- Vulnerability scanning
- Access control
- Audit logging

Security is integrated into daily operations.

Performance Optimization

The platform should optimize:

- API response time
- Database queries
- Image delivery
- Static asset caching
- Container startup time

Performance should be measured continuously.

Scalability Strategy

The MVP should scale without architectural redesign.

Growth stages:

Startup



Growing User Base



Multi-Team Organizations



Enterprise Customers

Cloud Run and Cloud SQL should support early growth with minimal operational complexity.

Cost Management

Operational costs should be monitored regularly.

Strategies include:

Auto-scaling

Resource optimization

Efficient storage usage

Log retention policies

Scheduled cleanup of unused assets

Infrastructure should remain economical during the MVP phase.

Disaster Recovery

Recovery planning includes:

Database restoration

Infrastructure recreation

Backup verification

Configuration recovery

Recovery procedures should be documented and tested periodically.

Operational Documentation

Maintain documentation for:

Deployment procedures

Rollback procedures

Environment configuration

Incident response

Infrastructure architecture

Documentation should evolve alongside the platform.

Success Criteria

The operations strategy succeeds when:

Deployments are predictable.

Services remain highly available.

Infrastructure scales with demand.

Failures are quickly detected.

Recovery procedures are effective.

Operational overhead remains low.

Conclusion

The HQ Platform Operations, Deployment & DevOps Blueprint establishes a reliable operational foundation for the platform.

By leveraging Google Cloud Platform for deployment while keeping the application architecture platform-independent, HQ can confidently meet hackathon objectives and continue evolving into a production-ready SaaS platform.